
Seeing the Signal: Image-Based Deep Learning for Limit Order Book Price Prediction

Candidates: 60236, 68951, 71066, 59816, 61125

ST311 Group Project
London School of Economics and Political Science

Abstract

We study price-direction prediction from limit order book (LOB) data on the FI-2010 benchmark under Setup 2. Existing strong models such as DeepLOB are engineered with (1×2) convolutions to pair prices with volumes before aggregating across price levels. We instead encode each 100-step LOB window as a four-channel image of shape $(4, 10, 100)$, with bid volume, bid price, ask volume, and ask price as separate channels and the ten price levels stacked along the height axis. A standard convolutional encoder is then followed by either a fully connected head or an LSTM. At horizon $k=50$, our CNN-MLP reaches a weighted F1 of 85.70 and our CNN-LSTM reaches 87.93, compared to 80.35 reported by DeepLOB. Most of the gain over DeepLOB is already obtained without any explicit temporal modelling, suggesting that the predictive structure of a 100-step LOB window is largely spatial.

1 Introduction

A limit order book is a real-time record of all outstanding buy (bid) and sell (ask) orders for a financial instrument, arranged by price level. At each level, the book records both a price and a volume, so that a snapshot of the top n levels on both sides yields $4n$ features per timestep [17]. Predicting short-term price movements from LOB data poses a challenge as they evolve rapidly across many price levels and is driven by a large number of market participants, producing data that is high-dimensional, non-stationary, and dominated by noise [17]. Through LOB literature, the standard prediction task is to classify the short-term direction of the mid-price, the average of the best bid and best ask prices, as upward, stationary, or downward [10]. The development of a standardised evaluation protocol in 2018 was critical to progress in this area: the FI-2010 dataset is the first publicly available benchmark for LOB mid-price forecasting, with approximately 4.5 million limit order events across five Finnish stocks over ten consecutive trading days on the Nasdaq Nordic exchange [10]. It is distributed already normalised (z-score), labelled, and sampled every 10 events. Two experimental setups are standard on this dataset: Setup 1 uses anchored forward cross-validation splitting by day, while Setup 2 trains on the first seven days and tests on the final three (Zhang et al., 2019, p. 6). [17].

The current best performance on FI-2010 is held by DeepLOB [17], whose convolutional component relies on carefully engineered (1×2) filters to pair prices with volumes and asks with bids before aggregating across levels. Recent work argues that LOB data is naturally suited to image-based representations through translation invariance, localised feature extraction, and hierarchical pattern recognition [1], and early evidence indicates such representations contain predictive signal [9]. However, no prior study has benchmarked image-based representations against established LOB models on a standard evaluation protocol.

This paper addresses that gap. We convert LOB snapshots into multi-channel heatmap images that separate prices and volumes into distinct channels and align bid and ask levels spatially, allowing conventional CNNs to learn from the LOB's structure without the specialised filter design required by

DeepLOB. We benchmark two image-based architectures (CNN-MLP, CNN-LSTM) against a reimplemented DeepLOB on FI-2010 under Setup 2 at $k=50$, using weighted F1-score for comparability with prior work.

2 Related work

2.1 Non-neural-network approaches

Before the emergence of deep learning, classical machine learning methods were mainly used for LOB prediction, which tended to rely on manually designed input features rather than allowing the model to learn useful representations directly from the data. The most notable of these is Kercheval and Zhang’s multiclass SVM framework whose work is foundational because it framed mid-price forecasting as a structured classification problem and introduced the feature engineering scheme (82 attributes organised into basic features, time-insensitive features and time-sensitive features) that was later adopted by Ntakaris et al. [10] to construct the FI-2010 benchmark dataset [7]. Within this same feature framework, Tran et al. [13] explored different multilinear methods on FI-2010 including LDA, MDA, and MCSDA. A shared limitation of all these non-neural methods is their dependence on static, handcrafted feature engineering that is not jointly optimised with the classification objective [17]. On the FI-2010 benchmark, these classical approaches achieved F1 scores between 35.88% and 47.87% at $k=10$ [17]. We will assess our non-neural models and our neural network models at $k=50$. We shall see that the neural networks perform significantly better than the non-neural models.

2.2 Neural network approaches

Tsantekidis et al. [15, 16] put forward the first neural network models applied to FI-2010, who evaluated an MLP, CNN, and LSTM under Setup 2. The key finding was that LSTM substantially outperformed the alternative feed-forward networks, thereby establishing the importance of temporal modelling for LOB prediction tasks. Moreover, later architectures from Passalis et al. [11] and Tran et al. [14] further improved the performance, with C(TABL) achieving the strongest pre-DeepLOB result of 78.44% F1 at $k=50$ [17]. However, these models treat the LOB as a flat numerical input, which is the representational assumption our work challenges.

Zhang et al.’s DeepLOB is the current best performing model on FI-2010, which achieves an F1 of 80.35% at $k=50$ under Setup 2 [17]. Its success is driven by two key design choices. First, the convolutional component uses (1×2) filters with a stride of (1×2) to pair prices with volumes at each level. The output feature map is subject to another (1×2) filter with (1×2) stride to pair ask and bid intra-level information. A (1×10) filter then aggregates across all levels [17]. Second, Inception Module captures dynamics over multiple timescales with learned weights, and an LSTM layer models longer-range temporal dependencies, all within approximately 60,000 parameters [17]. However, these architectural choices are specifically engineered around the flat numerical layout of the LOB input, motivating our question of whether restructuring the data itself, rather than designing custom filters to work around its layout, could allow a more standard architecture to achieve comparable results.

2.3 Image-based representations

The spatial structure of a LOB, with price levels along one axis and time along another, suggests a natural alternative to the flat numerical format: representing LOB snapshots as images. The first to explore this was Niño et al. [9], who encoded LOB and tick data into a four-channel image (ask volumes, bid volumes, order counts, and volume distributions) and then trained a two-layer CNN to classify price direction. They achieved 74.15% directional accuracy across 11 NYSE instruments [9]. This work demonstrates that image-based LOB representations contain predictive signal, however it has significant limitations: they used basic CNN architecture and their results were not benchmarked against any existing LOB-specific model or evaluated on a standard dataset such as FI-2010.

In recent literature, Backhouse et al. [1] represented LOB data as two-channel images (prices and volumes) and used diffusion models to generate synthetic order book states. They argue that convolutional models are well aligned to LOB images due to properties like translation invariance, localised feature extraction, and hierarchical pattern recognition [1]. However, their application

is LOB generation and simulation, not price movement classification, so the question of whether image-based representations are competitive for prediction remains open.

2.4 Our contribution

This paper directly addresses the gap outlined in this section. Although the theoretical suitability of image-based LOB representations for convolutional modelling has been argued [1] and early evidence shows that these representations contain predictive signal [9], no prior study has benchmarked an image-based approach against established LOB models on a standard evaluation protocol. In light of this we convert LOB snapshots into four-channel heatmap images that separate prices and volumes into distinct channels and align bid and ask levels spatially, which allows conventional CNNs to learn from the LOB’s structure without needing the specialised convolutional design required by DeepLOB. We benchmark two image-based architectures against DeepLOB on FI-2010 under Setup 2 at $k=50$, using weighted F1-score to account for class imbalance, providing the first controlled comparison between image-based and numerical LOB representations for mid-price prediction on a standard benchmark.

3 Data description

We use the FI-2010 benchmark dataset, adopting Setup 2 (training on days 1–7, testing on days 8–10) following Zhang et al. [17]. The dataset is downsampled by a factor of ten: each timestep in FI-2010 corresponds to a snapshot of the order book taken after every ten raw LOB events (order placements, cancellations, or executions) [10]. We use the decimal-precision normalised variant distributed via the DeepLOB GitHub repository¹, which applies the transform

$$x_{\text{DP}} = x \cdot 10^{-k}, \quad (1)$$

where k is the smallest integer such that $|x_{\text{DP}}| < 1$ [10].

Each timestep records 40 features representing the prices and volumes at the top ten levels on each side of the order book:

$$\left\{ p_a^{(i)}(t), v_a^{(i)}(t), p_b^{(i)}(t), v_b^{(i)}(t) \right\}_{i=1}^{10}. \quad (2)$$

3.1 Labelling

Following Ntakaris et al. [10], labels are derived from the mid-price

$$p_t = \frac{p_a^{(1)}(t) + p_b^{(1)}(t)}{2}. \quad (3)$$

The smoothed future mid-price is $m_+(t) = \frac{1}{k} \sum_{i=1}^k p_{t+i}$, where k denotes the prediction horizon, and the labels are determined by the percentage change

$$\ell_t = \frac{m_+(t) - p_t}{p_t} \quad (4)$$

thresholded by α into three classes: *Up* if $\ell_t > \alpha$, *Down* if $\ell_t < -\alpha$, and *Stationary* otherwise. We use $k=50$ throughout, the longest horizon for which Zhang et al. [17] report comparison results in their primary benchmark table.

At $k=50$, the training set is approximately balanced across the three classes (34.4% Up, 31.7% Stationary, 33.9% Down).

3.2 Biases

The labelling scheme in Equation (4) averages the next k mid-prices to define the future, but uses the single unaveraged value p_t to define the present. Since p_t serves as the anchor of the percentage change ℓ_t , small tick-level fluctuations in p_t can shift ℓ_t across the threshold α and flip the label between Up and Down at adjacent ticks, even when the broader price trajectory is unchanged [17, p. 4]. We adopt this scheme nonetheless, to ensure direct comparability with DeepLOB and all prior work evaluated on FI-2010

¹<https://github.com/zcakhhaa/DeepLOB-Deep-Convolutional-Neural-Networks-for-Limit-Order-Books>

3.3 Limitations

The FI-2010 only contains five stocks from a relatively illiquid Nordic exchange over a two-week window in 2010. The exchange’s relatively low liquidity means price patterns are more persistent and detectable, inflating model performance beyond what would be achievable in more competitive markets [12]. Additionally, given this data is from 2010, it is unlikely to reflect contemporary market behaviour. Prata et al. [12] finds that models trained on FI-2010 tend to overfit and under-perform when evaluated on more recent NASDAQ data. Despite the limitations, FI-2010 is a standard benchmark for LOB forecasting due to its public availability and the breadth of prior work using it, allowing for valuable comparison [12].

4 Methodology

4.1 LOB-as-image representation

The encoding intuition was the creation of a 4 channel heatmap encoding bid volume, bid price, ask volume and ask price respectively, each channel 10 LOB levels high and 100 timesteps wide. This was inspired by the method of Zhang who used (1×2) kernels with (1×2) strides to mix those same four pieces of information. By encoding this information in 4 channels, the CNN should be able to filter across the channels in a single step and learn their intralevel relationships, efficiently utilising the encoding structure of the data, rather than having to use specially picked filters and strides.

To construct the image representation, we extract sliding windows of 100 consecutive timesteps from the raw LOB features. Each window is then rearranged from its original flat format of $(100, 40)$ into a four channel image of shape $(4, 10, 100)$, where the width axis represents time and the height axis represents 10 price levels (Figure 1). Channel 0 and Channel 2 encode bid and ask volumes respectively, with ask volumes negated so that the two sides of the book differ in sign. Channel 1 and Channel 3 encode bid and ask prices respectively, centred on the window’s opening mid-price so that values represent deviations from a stable reference rather than absolute levels.

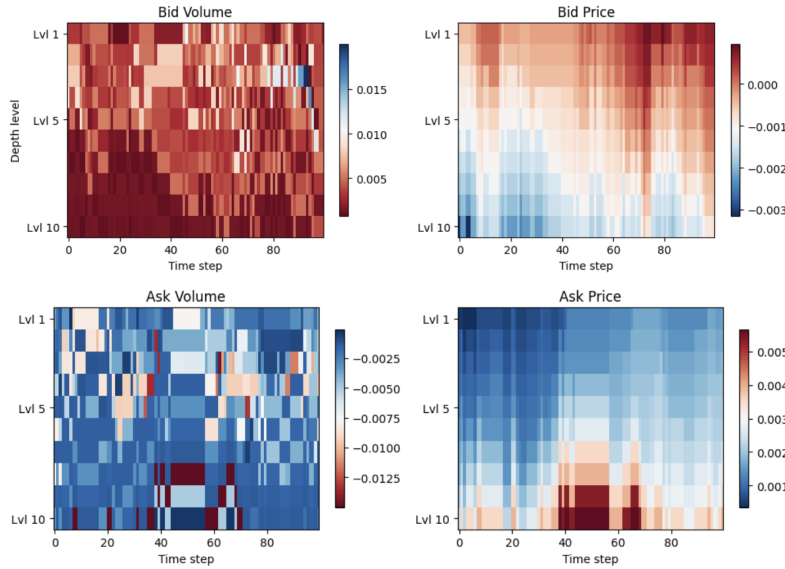


Figure 1: Example of a single LOB window encoded as a four-channel image. Each channel is 10 levels high and 100 timesteps wide.

Algorithm 1 LOB window to four-channel image

Require: LOB window $W \in \mathbb{R}^{T \times 4n}$ with $T = 100$ timesteps and $n = 10$ price levels per side
Ensure: Image $I \in \mathbb{R}^{4 \times n \times T}$ with channels (V_b, P_b, V_a, P_a)

- 1: Initialise $V_b, P_b, V_a, P_a \leftarrow \mathbf{0}^{n \times T}$
- 2: **for** $i = 0, 1, \dots, n - 1$ **do**
- 3: $c \leftarrow 4i$ $\triangleright$ starting column for level $i + 1$
- 4: $P_a[i, :] \leftarrow W[:, c]$ $\triangleright$ ask price
- 5: $V_a[i, :] \leftarrow -W[:, c + 1]$ $\triangleright$ negated ask volume
- 6: $P_b[i, :] \leftarrow W[:, c + 2]$ $\triangleright$ bid price
- 7: $V_b[i, :] \leftarrow W[:, c + 3]$ $\triangleright$ bid volume
- 8: **end for**
- 9: $m_0 \leftarrow (P_a[0, 0] + P_b[0, 0]) / 2$ $\triangleright$ opening mid-price
- 10: $P_a \leftarrow P_a - m_0$ $\triangleright$ centre ask prices
- 11: $P_b \leftarrow P_b - m_0$ $\triangleright$ centre bid prices
- 12: $I \leftarrow \text{stack}(V_b, P_b, V_a, P_a)$ along channel axis
- 13: **return** I

This layout turns predictive patterns such as bid-ask imbalances into spatially localised regions, aligning with the CNN’s inductive bias for detecting local structure.

4.2 Non-Neural baselines

To establish a non-NN floor for the comparison and to probe whether single-snapshot prediction is feasible at all, we evaluate two linear baselines that operate on the raw 40-dimensional snapshot at the final timestep of the window, $x = X_T \in \mathbb{R}^{40}$, standardised to zero mean and unit variance per feature.

4.2.1 Linear Discriminant Analysis

We include linear discriminant analysis (LDA) as a non-neural baseline because the multilinear discriminant methods that Tran et al. [13] apply to FI-2010 reduce to a plain LDA when the input is a flat vector, which ours is. LDA models each class as a Gaussian with its own mean and a covariance shared across classes, and assigns each input to the class with the highest posterior [6, Ch. 4]. Notably, the shared covariance produces linear decision boundaries, therefore making LDA a natural minimal benchmark. We fit it on the standardised training snapshots using the scikit-learn defaults, which estimate the class means and pooled covariance using their sample estimates.

4.2.2 Linear Support Vector

We also include a linear support vector classifier (LinearSVC) as a second non-neural baseline. This is because the multiclass linear SVM used by Kercheval and Zhang [7] to benchmark mid-price forecasting is the closest non-neural model in the LOB literature to what we are doing. LinearSVC finds the hyperplane that separates two classes with the largest possible margin. A regularisation constant C controls how much points on the wrong side of the margin are penalised [5, Section 3]. We use the scikit-learn implementation with $C=1.0$ and one-versus-rest for the three classes.

4.3 Neural Network Family

We propose two image-based neural architectures that share a common convolutional design but differ in how temporal information is handled: one uses a fully connected MLP, the other an LSTM. We describe the shared convolutional principles first, then each architecture in turn.

4.3.1 Convolutional Encoder

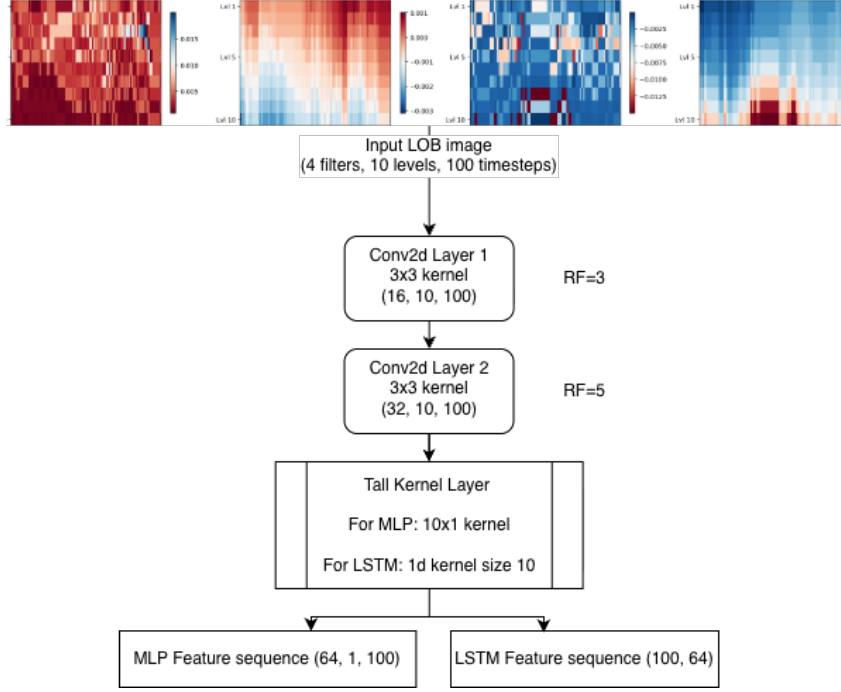


Figure 2: Convolutional encoder shared by the CNN-MLP and CNN-LSTM models.

Both architectures apply the same convolutional principles to the four-channel LOB image of shape (4, 10, 100), where each channel encodes the prices and volumes for bids and asks, height represents the 10 price levels, and width represents 100 consecutive timesteps. The extracted features are then passed to either a fully connected MLP head or an LSTM module for classification.

We select 3×3 kernels as they provide the minimum receptive field for detecting cross-level, cross-temporal patterns, while avoiding the smearing that larger kernels would introduce. Two layers give a receptive field $RF = 5$, sufficient for capturing bid/ask imbalance across adjacent levels. We avoid downsampling to prevent loss of positional information, since price levels closer to the best bid and ask carry more predictive signal than deeper levels [17]. Instead, all convolutional layers use stride 1 and padding 1 to preserve the full (10, 100) spatial resolution.

Both architectures end their convolutional stack with a tall-kernel layer that spans all 10 levels in a single operation, collapsing the level axis into a learned weighted combination across depth. This allows the CNN to learn explicit per-level weightings rather than relying on downstream layers to disentangle level importance from a flattened vector. The CNN-MLP implements this as a 2D convolution with kernel (10, 1), producing a (64, 1, 100) feature map. The CNN-LSTM applies an equivalent 1D convolution with kernel size 10 independently at each timestep, producing a 64-dimensional feature vector per timestep.

4.3.2 CNN-MLP

After the convolutional blocks, the flattened feature vector $y^{(0)} \in \mathbb{R}^{6400}$ is passed through one hidden layer and a linear output layer:

$$z^{(1)} = W^{(1)}y^{(0)} + b^{(1)}, \quad y^{(1)} = \text{ReLU}(z^{(1)}), \quad (5)$$

$$z^{(2)} = W^{(2)}y^{(1)} + b^{(2)}, \quad \hat{p} = \text{softmax}(z^{(2)}), \quad (6)$$

where $\text{ReLU}(z) = \max(0, z)$ is applied elementwise. The parameters are $W^{(1)} \in \mathbb{R}^{128 \times 6400}$, $b^{(1)} \in \mathbb{R}^{128}$, $W^{(2)} \in \mathbb{R}^{3 \times 128}$, $b^{(2)} \in \mathbb{R}^3$, and $\hat{p} \in \mathbb{R}^3$ gives the predicted class probabilities used in the cross-entropy loss (Appendix C).

The MLP head, shown in Figure 3a, consists of two linear layers with ReLU and dropout. The flattened dimension is 64×100 , a flat representation of the time-resolved per-level weightings. The hidden layer compresses this to 128 dimensions; dropout regularises to mitigate overfitting on the relatively small FI-2010 training set. The final linear layer projects to 3 logits for the {down, stay, up} classification.

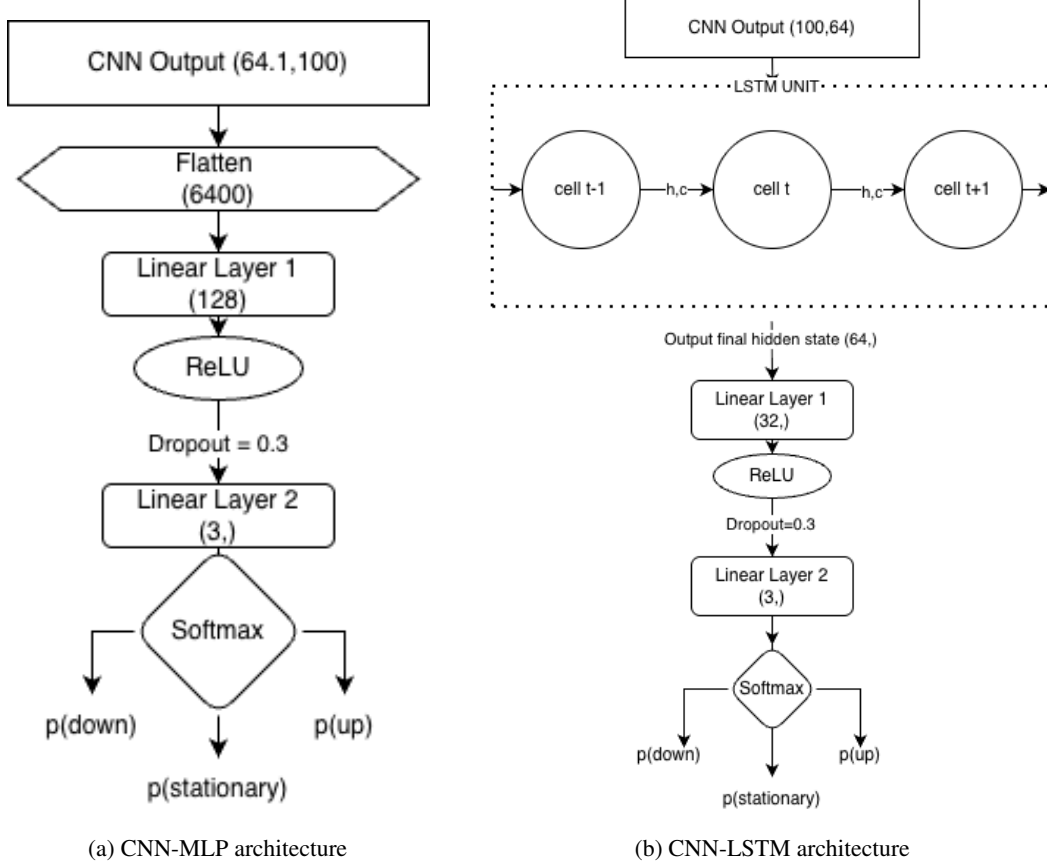


Figure 3: Architecture diagrams for the two image-based models.

4.3.3 CNN-LSTM

To explicitly model temporal dependencies in the LOB feature stream, the CNN encoder produces a sequence of 64-dimensional per-timestep feature vectors which are then consumed by a recurrent module, shown in Figure 3b. A vanilla RNN would struggle to capture long-range dependencies due to vanishing and exploding gradients, motivating the use of gated architectures. The LSTM addresses this by introducing an explicit memory cell with multiplicative input, forget, and output gates. At each timestep, four gating computations are done in parallel from $x_t \in \mathbb{R}^{64}$ and $h_{t-1} \in \mathbb{R}^{64}$:

$$\begin{aligned}
 i_t &= \sigma(W_{ii}x_t + W_{hi}h_{t-1} + b_i) && \text{input gate} \\
 f_t &= \sigma(W_{if}x_t + W_{hf}h_{t-1} + b_f) && \text{forget gate} \\
 o_t &= \sigma(W_{io}x_t + W_{ho}h_{t-1} + b_o) && \text{output gate} \\
 \tilde{c}_t &= \tanh(W_{ig}x_t + W_{hg}h_{t-1} + b_g) && \text{candidate cell}
 \end{aligned} \tag{7}$$

Then the memory cell internal state c_t and hidden state h_t are updated:

$$\begin{aligned}
 c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
 h_t &= o_t \odot \tanh(c_t)
 \end{aligned} \tag{8}$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function and $\odot$ is the dot product.

The LSTM processes the 100-step sequence and the final hidden state $h_T \in \mathbb{R}^{64}$ is passed through a 32-unit fully connected layer with ReLU and dropout, followed by a linear projection to 3 class logits, from which class probabilities are obtained via softmax (Appendix C).

5 Experiments

5.1 Training setup

To guarantee reproducibility, all experiments were initialised with a fixed random seed of 42 across both NumPy and PyTorch. Following the DeepLOB experimental setup, we utilised the first seven days of the FI-2010 dataset for training, isolating a 20% chronological split from the end of this period for validation.

Because our methodology extracts sliding windows of 100 consecutive timesteps, a standard chronological split introduces a minor data leakage risk at the boundary between the training and validation sets. While this overlap can slightly inflate validation accuracy during the early-stopping monitoring phase, it does not compromise our final evaluation. This is because final testing is conducted exclusively on strictly out-of-sample data (days 8–10) that shares no overlapping windows with the training period.

To document our computational footprint, all training and evaluation runs were executed using a single NVIDIA T4 GPU.

5.2 Hyperparameter Tuning and Training Regime

The non-neural baselines were fit by closed-form solution (LDA) or convex optimisation (LinearSVC) and therefore do not involve a learning rate, mini-batches, or early stopping.

We tune four hyperparameters: learning rate, batch size, dropout, and weight decay. Due to GPU constraints, these are tuned in two stages rather than as one full grid. Stage 1 searches over learning rate and batch size. Next, Stage 2 fixes the best learning rate and batch size from Stage 1, then searches over dropout and weight decay. Learning rate and batch size are tuned first because the learning rate is central to deep network optimisation [2, Section 19.3.1], and because batch size affects the noise of each gradient update. After this, Dropout and weight decay are tuned in Stage 2 because the amount of regularisation required depends on the training regime. We use grid search in both stages.

In Stage 1 we try learning rates of 5×10^{-4} , 10^{-3} , and 2×10^{-3} against batch sizes of 32, 64, and 128, giving nine runs per model. In Stage 2 we try dropout values of 0.0, 0.1, 0.3 and 0.5 against weight decay values of 0, 10^{-5} , 10^{-4} , 5×10^{-4} , and 10^{-3} , giving 25 runs per model. Each run is ranked by its best validation weighted F1, with validation loss as a tiebreaker.

All neural models were trained using the AdamW optimiser and the standard softmax cross-entropy loss (see Appendix C). After tuning, the image-based models were retrained using their selected hyperparameters for a maximum of 100 epochs. To prevent overfitting, early stopping was employed with a patience of 7 epochs by monitoring the validation loss.

For the DeepLOB baseline, we report the results published by Zhang et al. [17] under the same Setup 2 protocol at $k=50$.

5.3 Evaluation protocol

To rigorously evaluate the generalisability of the models, final testing was conducted on three datasets representing consecutive, unseen trading days (days 8, 9 and 10). Model performance for the $k=50$ prediction horizon was quantified using overall accuracy, alongside weighted precision, recall and F1-scores. We prioritise the weighted F1-score to maintain direct comparability with DeepLOB and prior work evaluated on FI-2010, which uniformly report this metric.

For each class c , precision and recall are defined as:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \quad (9)$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (10)$$

where TP_c , FP_c , and FN_c represent the true positives, false positives, and false negatives for class c , respectively. The F1-score for each class is then computed from (9) and (10) as follows:

$$F1_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (11)$$

The final weighted F1-score across all classes C aggregates the per-class scores from (11):

$$\text{Weighted } F1 = \sum_{c \in C} W_c F1_c \quad (12)$$

where W_c represents the proportion of actual instances belonging to class c . Results were aggregated to report both per-day performance and the mean performance across all three test days to ensure consistency.

6 Results

Table 1 reports the test-set performance of our two image-based models alongside the two non-neural baselines and previously published Setup 2 results at $k=50$. Each of our models was retrained on the full first seven days using the configuration selected in Appendix D and evaluated on the held-out test days 8-10. We extract sliding windows within each of days 8, 9 and 10 separately, concatenate the three into a single test set, and report the weighted metrics on the pooled set. Per-day F1 scores are given in Table 2 as a consistency check.

Table 1: Experiment results for the FI-2010 dataset at prediction horizon $k=50$, Setup 2.

Model	Accuracy	Precision	Recall	F1
<i>Non-neural and shallow baselines</i>				
SVM [16]	–	60.30	46.05	49.42
MLP [16]	–	67.14	55.21	55.95
<i>Neural network approaches</i>				
CNN-I [15]	–	67.12	55.58	59.44
LSTM [16]	–	60.03	68.50	61.43
B(TABL) [14]	69.54	69.12	68.84	68.84
C(TABL) [14]	74.81	74.58	74.27	74.32
DeepLOB [17]	80.51	80.38	80.51	80.35
<i>Our Results</i>				
LDA	41.63	44.95	41.63	42.25
LinearSVC	41.11	44.80	41.11	41.70
CNN-MLP	85.75	85.79	85.75	85.70
CNN-LSTM	88.02	88.36	88.02	87.93

Both of our models beat DeepLOB on weighted F1 by several F1 points. The CNN-MLP, with no temporal mechanism beyond what the convolutional stack provides, sits 5.35 F1 points above DeepLOB (85.70 vs 80.35). The CNN-LSTM extends this to 7.58 points (87.93 vs 80.35). This supports our central claim that the four-channel image encoding makes the predictive structure of the LOB directly accessible to a standard CNN, without requiring specialised architectural choices for the flat numerical layout.

The CNN-LSTM improves on the CNN-MLP by 2.23 F1 points. This is a real gain but a small one, suggesting that most of the predictive signal in a 100-step window is already picked up by the spatial structure of the order book through the convolutional encoder. Explicit temporal modelling adds an extra layer of information on top, but is not where the bulk of the performance comes from.

LDA and LinearSVC, applied to the 40-features snapshot at the final timestep of each window, reach an F1 of 42.25 and 41.70 respectively. The gap of around 44 F1 points to our CNN-MLP shows that

the predictive signal is not in the final snapshot alone, it lives in how prices and volumes evolve across the window and in non-linear interactions across price levels, neither of which a linear classifier on a single snapshot can capture.

Table 2: Per-day weighted F1 scores on test days 8–10.

Model	Day 8	Day 9	Day 10
LDA	42.59	41.01	42.54
LinearSVC	41.64	40.34	42.79
CNN-MLP	86.41	85.53	84.78
CNN-LSTM	88.93	87.48	86.94

Both image-based models stay consistent across the three test days. Within-model variation never exceeds 3 F1 points across the three days.

7 Conclusion

In this project we presented an image-based approach to price movement prediction with limit order book data. By encoding the prices and volumes of buy and sell orders into four-channel heatmap images, we reframe price movement prediction as a computer vision problem, directly accessible to CNN architecture. Using a CNN to extract spatial features of the order book, we tested our approach, passing its output to a fully connected MLP as well as an LSTM to capture temporal dependencies. On the FI-2010 benchmark at $k=50$ under Setup 2, our CNN-LSTM achieves a weighted F1 of 87.93%, outperforming DeepLOB by 7.58 points. Our CNN-MLP also beats DeepLOB at 85.70% F1, suggesting that the image encoding itself captures meaningful structure in the order book even without explicit temporal modelling, and that the recurrent layer adds a further but smaller gain on top.

However, there are three main limitations to these findings, which arose from compute constraints and the availability of open-source data. Firstly, we evaluated on a single benchmark, the FI-2010. This consists of somewhat illiquid Nordic stocks and prior work has shown that the models tuned on it tend to lose substantial performance when transferred to more recent and more liquid markets such as NASDAQ [12]. Therefore, our reported gains over DeepLOB must be interpreted as evidence on this benchmark, as opposed to a general claim. Secondly, we trained and evaluated at a single prediction horizon ($k=50$). If we were to repeat the experiment across horizons, this would reveal whether the gain over DeepLOB is stable or specific to $k=50$. Third, results are reported from single training runs per configuration, so we cannot quantify variance across seeds.

A natural extension of our paper would be to swap the CNN encoder for a vision transformer, specifically in ViT-LOB [8]. This paper has found that LOB snapshots are usefully treated as images, therefore ViT models, that learn from image patches, are a natural next step. Moreover, a second extension would be to apply attention over both the feature axis and the time axis at once, following TLOB [4]. This would replace our CNN-LSTM stack with a single model that learns spatial and temporal patterns together. While this was outside the scope of this project given compute budget and the lack of openly available data, the pipeline should be tested on more recent and more liquid order book data, such as NASDAQ stocks, where existing models lose performance [4].

References

- [1] Backhouse, A., Li, K., Foerster, J., Calinescu, A. and Zohren, S. (2025). Painting the market: generative diffusion models for financial limit order book simulation and forecasting.
- [2] Bengio, Y. (2012). Practical recommendations for gradient-based training of deep architectures. In: G. Montavon, G.B. Orr and K.-R. Müller, eds., *Neural Networks: Tricks of the Trade*, 2nd ed., Lecture Notes in Computer Science, vol. 7700. Berlin, Heidelberg: Springer, pp. 437–478.
- [3] Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, pp. 281–305.
- [4] Berti, L. and Kasneci, G. (2025). TLOB: A novel transformer model with dual attention for stock price trend prediction with limit order book data. *arXiv preprint arXiv:2502.15757*.
- [5] Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), pp. 273–297.
- [6] Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer. Springer Series in Statistics.
- [7] Kercheval, A.N. and Zhang, Y. (2015). Modelling high-frequency limit order book dynamics with support vector machines. *Quantitative Finance*, 15(8), pp. 1315–1329. doi:<https://doi.org/10.1080/14697688.2015.1032546>.
- [8] Liu, Z., Sham, C.-W., Ma, L. and Fu, C. (2024). ViT-LOB: Efficient vision transformer for stock price trend prediction using limit order books. In: *2024 10th International Conference on Applied System Innovation (ICASI)*. IEEE, pp. 436–438.
- [9] Niño, J., Arévalo, A., Leon, D., Hernandez, G. and Sandoval, J. (2018). Price Prediction with CNN and Limit Order Book Data.
- [10] Ntakaris, A., Magris, M., Kannianen, J., Gabbouj, M. and Iosifidis, A. (2018). Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8), pp. 852–866. doi:<https://doi.org/10.1002/for.2543>.
- [11] Passalis, N., Tefas, A., Kannianen, J., Gabbouj, M. and Iosifidis, A. (2020). Temporal Bag-of-Features Learning for Predicting Mid Price Movements Using High Frequency Limit Order Book Data. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 4(6), pp. 774–785. doi:<https://doi.org/10.1109/tetci.2018.2872598>.
- [12] Prata, M., Masi, G., Berti, L., Arrigoni, V., Coletta, A., Cannistraci, I., Vyetrenko, S., Velardi, P. and Bartolini, N. (2023). LOB-Based Deep Learning Models for Stock Price Trend Prediction: A Benchmark Study. [online] Available at: <https://arxiv.org/abs/2308.01915> [Accessed 21 Apr. 2026].
- [13] Tran, D., Gabbouj, M. and Iosifidis, A. (2017). Multilinear Class-Specific Discriminant Analysis. [online] Available at: <https://arxiv.org/pdf/1710.10695> [Accessed 21 Apr. 2026].
- [14] Tran, D.T., Iosifidis, A., Kannianen, J. and Gabbouj, M. (2019). Temporal Attention-Augmented Bilinear Network for Financial Time-Series Data Analysis. *IEEE Transactions on Neural Networks and Learning Systems*, 30(5), pp. 1407–1418. doi:<https://doi.org/10.1109/tnnls.2018.2869225>.
- [15] Tsantekidis, A., Passalis, N., Tefas, A., Kannianen, J., Gabbouj, M. and Iosifidis, A. (2017). Forecasting Stock Prices from the Limit Order Book using Convolutional Neural Networks.
- [16] Tsantekidis, A., Passalis, N., Tefas, A., Kannianen, J., Gabbouj, M. and Iosifidis, A. (2017). Using Deep Learning to Detect Price Change Indications in Financial Markets.
- [17] Zhang, Z., Zohren, S. and Roberts, S. (2019). DeepLOB: Deep Convolutional Neural Networks for Limit Order Books. *IEEE Transactions on Signal Processing*, 67(11), pp. 3001–3012. doi:<https://doi.org/10.1109/tsp.2019.2907260>.

Declarations

A Statement of Individual Contributions

Candidate 60236 (20%): In terms of the report, I wrote the following sections: Introduction, Related Work (Non-NN approaches, NN approaches, Image based representations and Our Contribution), the Non-NN Section in Methodology (LDA and LinearSVC), the Hyperparameter tuning in Experiments section, the limitations and future work in the Conclusion, Bibliography, AI usage statement and the Appendix. For the code, I implemented the hyper parameter tuning and the LDA/LinearSVC non-NN baselines.

Candidate 68951 (20%): I worked on Data Description, Methodology(equations, algorithm box, schematic diagrams), Results, Abstract, and contributed ideas to the methodology writeup. Re-ran the code after fixing some bits and editing the testing to evaluate models on the pooled test set across days 8-10.

Candidate 71066 (20%): I theoretically justified and designed the two specialised neural network models, after designing the 4 channel heatmap to which the models were applied, justifying the conversion of LOB data to images, and created the algorithm box. I wrote part of the methodology and the results, and did most of the model training.

Candidate 61125 (20%): I worked on initial model design and experimentation, coding the main training loop and initial variants of our models. In the report I worked on the conclusion and contributed to methodology section designing all of the diagrams. I also researched the dataset and wrote the limitations section on data description

Candidate 59816 (20%): For this project, I proposed the initial project idea of representing limit order book snapshots as two-channel heatmap images for CNN classification. For the code, I built the original image construction pipeline for the FI-2010 dataset, which was later refined. I wrote the Experiments section (5) and refined the Methodology section (4). I also prepared the presentation slides.

B AI Usage Statement

This paper closely followed the *International Conference on Machine Learning 2023*² policy and the *ST311: AI Policy* about declaring the use of AI. In this paper, Generative AI (namely ChatGPT, Claude and GitHub Copilot) was used for the following:

- **Coding assistance:** Across the group, AI was used to assist with writing code patterns not directly provided in the lectures and code that was specific to this project context, such as downloading and manipulating the raw LOB data. It was also used to help debug errors by explaining the errors and suggesting possible fixes or workarounds, which were then integrated by the group. We also used it to improve the formatting of outputs, such as the hyperparameter tuning and results tables of the pooled evaluation.
- **Report assistance:** In this report, AI was used mainly for grammar/spell checks and to polish text to improve clarity. Specifically, AI was not used to produce text, but rather it helped us edit text that we wrote ourselves. We also used AI to help check that our formatting followed NeurIPS guidelines. Moreover, we used AI for limited suggestions on report structure: which content to place in the appendix, how to best to organise subsections for readability purposes. Lastly, *MyBib* was used for formatting the Bibliography.

Appendix

C Softmax cross-entropy loss

For a sample with logit vector $z \in \mathbb{R}^3$ produced by the model, the predicted class probabilities are obtained via the softmax

$$\hat{p}_k = \frac{\exp(z_k)}{\sum_{j=1}^3 \exp(z_j)}, \quad k \in \{1, 2, 3\}. \quad (13)$$

²<https://icml.cc/Conferences/2023/llm-policy>

The per-sample cross-entropy loss is

$$\ell(y, \hat{p}) = - \sum_{k=1}^3 y_k \log \hat{p}_k = - \log \hat{p}_c, \quad (14)$$

where $y \in \{0, 1\}^3$ is the label vector with exactly one entry equal to 1. The simplification to $-\log \hat{p}_c$ follows because all terms with $y_k = 0$ vanish, leaving only the term for the true class. The total training loss averages this per-sample loss over the N training windows:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, \hat{p}^{(i)}; \theta), \quad (15)$$

where θ denotes the model parameters.

D Hyperparameter tuning

This gives the configurations selected by the hyperparameter tuning described in Section 5.2. The following search ranges were performed across the two stages:

Table 3: Search ranges for each tuning stage.

Stage	Hyperparameter	Values
1	learning rate	$\{5 \times 10^{-4}, 10^{-3}, 2 \times 10^{-3}\}$
1	batch size	$\{32, 64, 128\}$
2	dropout (MLP)	$\{0.1, 0.3, 0.5\}$
2	dropout (LSTM)	$\{0.0, 0.1, 0.3, 0.5\}$
2	weight decay	$\{0, 10^{-4}, 10^{-3}\}$

The results and selected configurations for each model following the tuning are as follows:

Table 4: Hyperparameters selected for final training, with Stage 2 validation metrics.

Model	lr	batch	dropout	wd	val loss	val F1
CNN-MLP	0.001	32	0.3	0	0.4176	0.8475
CNN-LSTM	0.002	64	0.0	0.0001	0.3704	0.8677

Five of the 78 trials failed to train, terminating at epoch 1 with a validation F1 of 0.15. All five were at corners of the search space that combined a high learning rate with a small batch size or extreme regularisation. These trials are in the supplementary CSVs but were not candidates for selection. The following settings were fixed across all runs:

- Optimiser: Adam with default $\beta_1=0.9$, $\beta_2=0.999$.
- Loss: cross-entropy.
- Random seed: 42.
- Train/validation split: 80/20 of the FI-2010 training partition.
- Early stopping on validation loss, with patience of 3 in Stages 1 and 2 and patience of 7 in final training.
- Epoch caps: 20 in Stage 1, 25 in Stage 2, 100 in final training. Neither of the two selected models reached the final-training cap.
- At the end of each run, the model state with the lowest validation loss is restored before reporting metrics.